



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Sesión 1: Programación Funcional

Semillero en Ciencias de la Computación

Joel Ostos Juan Pedrozo

Abril 24, 2026

1. Un vistazo a la programación funcional y sus alrededores

Paradigmas de programación

- POO

- Programación Imperativa

- Programación Declarativa

- Cálculo Lambda

Un poco de historia

Qué es la programación Funcional

- Efectos secundarios

- Inmutabilidad

- Funciones Puras

2. Ejemplos Prácticos en Clojure

1. Un vistazo a la programación funcional y sus alrededores

Paradigmas de programación

POO

Programación Imperativa

Programación Declarativa

Cálculo Lambda

Un poco de historia

Qué es la programación Funcional

Efectos secundarios

Inmutabilidad

Funciones Puras

2. Ejemplos Prácticos en Clojure



1. Orientado a Objetos.
2. Imperativo.
3. Declarativo.
4. Funcional.



Antes, la programación era mayormente procedural. Los programas eran una larga lista de instrucciones. POO tiene como pilares:

1. Encapsulamiento
2. Abstracción
3. Herencia
4. Polimorfismo



Agrupar datos y comportamiento en una sola unidad: el **objeto**.

- El objeto expone solo lo necesario
- El resto queda *oculto* al exterior

`persona.nombre` vs `persona._id_interno`

Modelar el mundo real ignorando los detalles irrelevantes.

- Un Auto tiene `acelerar()` — no importa cómo funciona el motor
- Se trabaja con *qué hace*, no *cómo lo hace*

Una clase puede *heredar* atributos y métodos de otra.

- Perro y Gato heredan de `Animal`
- Favorece la reutilización de código

`Animal` → Perro, Gato, Ave ...



El mismo mensaje, distintos comportamientos según el objeto.

- `animal.hablar()` → "*Guau*" o "*Miau*" según sea el caso
- El código no necesita saber *qué tipo* de objeto es



```
public static int calculateScore(String word) {  
    int score = 0;  
    for(char c : word.toCharArray()) {  
        score++;  
    }  
    return score;  
}
```

La programación imperativa se enfoca en el **como** debe ser computado el resultado.



```
public static int wordScore(String word) {  
    return word.length();  
}
```

La programación declarativa se enfoca en qué se necesita hacer, no como. Generalmente es más sencilla de leer que la programación imperativa.

Alan Turing

Máquina de estados,
instrucciones secuenciales

→ Programación **Imperativa**

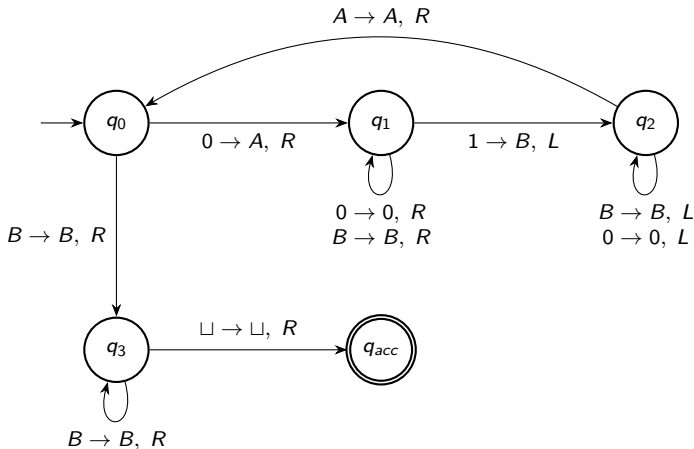
Alonzo Church

Cálculo Lambda,
transformación de funciones

→ Programación **Funcional**

Ambos modelos son equivalentes en poder computacional, pero proponen
maneras distintas de pensar.

Máquina de Turing: ¿Es 0^n1^n ?



El mismo problema en Cálculo Lambda

```
(defn count-ady [s]
  (->> (partition-by identity s)
        (map (juxt first count)))))

(defn fmt-ady [s]
  (let [coll (count-ady s)]
    (when (= (count coll) 2)
      (let [[[_ n1] [_ n2]] coll]
        (= n1 n2))))))
```

Diferencia fundamental

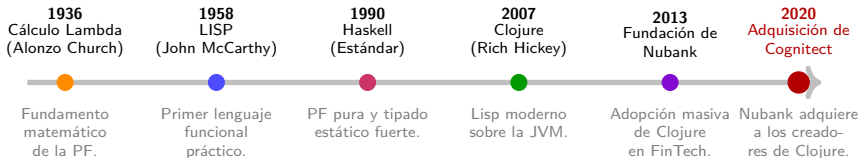
La MT *recorre* una cinta modificando símbolos paso a paso. El cálculo lambda **transforma** la lista con funciones — sin mutación, sin estados.

- **¿Qué es?:** Un sistema matemático formal para expresar la computación basado en la **abstracción de funciones**.
- **Influencia:** Es la base teórica de todos los lenguajes funcionales (Lisp, Haskell, Clojure).

Los tres pilares del Cálculo Lambda

1. **Variables:** x
2. **Abstracción:** $\lambda x.M$ (Define una función que toma x y devuelve M)
3. **Aplicación:** $(M N)$ (Aplica la función M al argumento N)

En Clojure, una función anónima (`fn [x] (* x x)`) es una abstracción lambda.



Dato relevante

Gracias al cálculo lambda se puede demostrar que algún lenguaje es turing completo, como lo es el caso de Clojure y Haskell.

Entonces, ¿Qué es la programación funcional?



Programación mediante **transformaciones**, no mediante órdenes.

1. **Cero efectos secundarios (Side Effects)**
2. **Inmutabilidad**
3. **Funciones Puras**

¿Qué son los efectos secundarios?



Cuando el único resultado observable de una función es el valor que retorna decimos que la función no tiene efectos secundarios. Si su función cambia alguna variable global o local más allá del valor que retorna entonces esta posee efectos secundarios.

```
public class EjemploSideEffect {  
  
    static int contador = 0;  
  
    public static void foo( ) {  
        contador++; // side effect  
    }  
  
    public static void main(String[] args) {  
        foo();  
        System.out.println(contador);  
    }  
}
```



1. Los datos no cambian después de crearse.
2. En lugar de modificar un valor se crea uno nuevo
3. Hace que el comportamiento del código sea más predecible pues se evitan los **side effects**



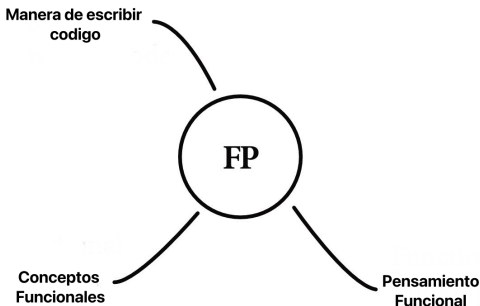
1. Misma entrada $\rightarrow$ Misma salida.
2. No tienen efectos secundarios.
3. Son predecibles y fáciles de probar.

```
int suma(int a, int b) {  
    return a + b;  
}
```

¿Qué tan útil es aprender programación funcional?



1. Es un estilo de escribir código en cualquier lenguaje.
2. Es como un cambio de chip en la manera de pensar en programación declarativa.



1. Un vistazo a la programación funcional y sus alrededores

Paradigmas de programación

POO

Programación Imperativa

Programación Declarativa

Cálculo Lambda

Un poco de historia

Qué es la programación Funcional

Efectos secundarios

Inmutabilidad

Funciones Puras

2. Ejemplos Prácticos en Clojure

Aplicación en Clojure

- **Lisp:** Dialecto potente y expresivo con un enfoque funcional puro.
- **Alojado:** Diseñado para ejecutarse sobre plataformas existentes.
 - Principalmente la **JVM** (Java Virtual Machine).
 - También compila a JavaScript y CLR (.NET).

Compilación en la JVM

El compilador (`clojure.jar`) transforma el código fuente directamente en **Java Bytecode**, aprovechando el recolector de basura y el manejo de hilos de Java.

- **LISt Processing**
- **Homoiconicidad:** Código igual que estructuras de datos
- **Prefix notation**

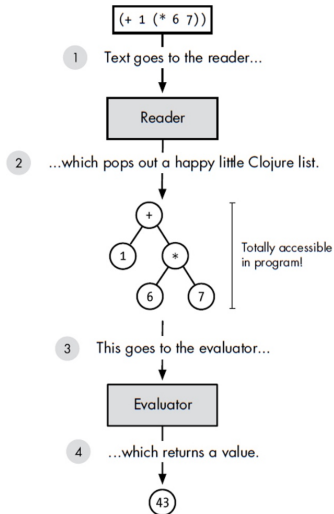
Anatomía de una Expresión

- Estructura: (operador argumento1 argumento2 ...)

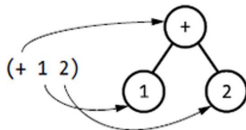
```
;; Suma simple: (+ 1 2 3) -> 6
```

```
(defn saludar [nombre]  
  (str "Hola, " nombre))
```

1. **Reader:** Texto a estructuras de datos
2. **Data Structures**
3. **Evaluator**
4. **Result**

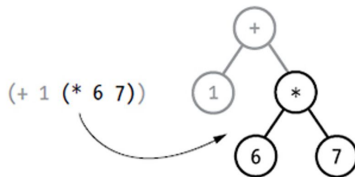


The first element is treated as the root.



1. Las funciones anidadas se leen de paréntesis interiores a exteriores

Subsequent elements are treated as branches.



Nested lists are treated as nested trees.

- Se utiliza la macro `defn`.
- Los parámetros se definen dentro de un **vector** `[]`.
- El cuerpo de la función devuelve automáticamente el valor de la última expresión.

Sintaxis Básica

```
(defn saludar []  
  (println "Hola Mundo"))
```

```
(defn calcular-area [radio]  
  (* 3.14 (* radio radio)))
```

```
(defn promedio [a b]  
  (/ (+ a b) 2))
```



- **¿Qué es?:** *Read-Eval-Print Loop*. Una interfaz para interactuar con un proceso de Clojure en ejecución.
- **Filosofía:** Permite modificar el programa en vivo incluso en producción, similar a una conexión SSH.
- **Feedback Inmediato:** Ideal para experimentar con sintaxis y lógica sin reiniciar la aplicación.

Uso del REPL

```
user=> (+ 1 2 3 4)
10
```

```
user=> (first [1 2 3 4])
1
```

```
(do
  (println "Evaluando...")
  (+ 1 3))
; => Evaluando...
4
```



El caso de println

```
user=> (println "Hola semillero!")  
; Hola semillero! <--- SIDE EFFECT  
nil <--- VALOR DE RETORNO
```

¿Por qué es un Side Effect?

Porque println no devuelve el texto; su trabajo es **modificar el estado de la pantalla**. En Clojure, println siempre devuelve nil como valor.



- **4Clojure (Interactive Problems):** La mejor plataforma para aprender la sintaxis y el pensamiento funcional mediante retos progresivos.
- **¿Qué encontrarás?:** Desde cómo funcionan las listas hasta la manipulación avanzada de datos.

Ejercicios Interactivos

Puedes encontrar y resolver ejemplos prácticos en:

<https://4clojure.oxal.org/>

Gracias!